Figure 2 is a high-level presentation of Kubernetes architecture.

The Kubernetes cluster consists of one or more nodes, which may be physical or virtual machines. Each node can run several pods. A pod is a group of containers. Each pod gets an ephemeral IP address that is known as *cluster-ip* address. This address is local to the cluster and visible to all other pods across it. In other words, the pods within the cluster can reach out to each other using the *cluster-ip* address.

Normally, a pod consists of only one application container that runs a microservice. Besides this, a pod may also run several other infrastructure containers that take on tasks such as monitoring, logging, etc.

A deployment unit in Kubernetes consists of a set of such pods. This set is known as a replica-set. For example, you can create a deployment unit for *AddService* in such a way that three pods are scheduled in the cluster with each pod running an *AddService* container. If any of the pods crash for whatever reason, Kubernetes schedules another pod on the cluster in such a way that three pods of *AddService* are always running. Note that the pods of a replica-set do not necessarily run on a single node.

Though this is sufficient for the containers on different pods/nodes to collaborate with each other, it is very cumbersome for a pod to address another pod based on an ephemeral address. To solve this problem, we can create a front-end to each of the replica-sets. Such a front-end is called a service. Each service is exposed with an address that is not ephemeral. The address is called *node-port* if it is made visible only within the cluster or *external-ip* if exposed outside the cluster. A service can also be configured with a load balancer so that the incoming calls can be routed to the end-points (pods) in a fairly balanced manner.

This is Kubernetes in a nutshell. There are several tools available in the market to set up a Kubernetes cluster. Minikube is one such tool that helps in setting up single-node clusters. The instructions given below can be followed to set up the Minikube cluster on an Ubuntu machine that has Docker engine running.

Download Minikube distribution.

```
$ curl -LO https://storage.googleapis.com/minikube/releases/
latest/minikube-linux-amd64
```

Give the following command to install Minikube:

```
$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Create a group named *docker* and add the user:

```
$ sudo usermod -aG docker <user> && newgrp docker
```

Start the cluster:

```
$ minikube start
```

Create this handy alias:

```
$ alias kubectl="minikube kubectl --"
```

A single-node Kubernetes cluster is now up and running on the local machine.

Deploying MySQL on Kubernetes

Let us deploy a replica-set consisting of just one pod of MySQL. The service is exposed by the name *mysqldb*. Other pods must use this name in order to access the database service. The port 3306 is exposed only within the cluster. We don't want any one from outside the cluster to log in to our database server. The deployment also mandates to create a schema by the name *glarimy* and to use a mounted volume.

```
apiVersion: v1
kind: Service
metadata:
  name: mysqldb
spec:
  ports:
  - port: 3306
  selector:
    app: mysqldb
  clusterIP: None
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysqldb
spec:
  selector:
    matchLabels:
      app: mysqldb
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysqldb
    spec:
      containers:
     - image: mysql:5.6
        name: mysqldb
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: admin
```